

AIVOA — AI-Powered Customer Complaint Management System

An AI-powered Customer Complaint Management System for pharmaceutical manufacturing (API & FDF Quality Assurance), built for the AIVOA AI Product Engineer internship assignment.

The system uses a conversational AI Copilot to ingest complaints via natural language text or PDF upload, automatically extracting and structuring complaint data, classifying defects, assessing risk, and generating CAPA recommendations — all without manual form entry.

Tech Stack

Layer	Technology
Frontend	React 18 + Redux Toolkit + Vite
Backend	Python + FastAPI
AI Agent Framework	LangGraph (StateGraph with conditional routing)
LLMs	Groq API — openai/gpt-oss-20b (primary), openai/gpt-oss-120b (risk assessment)
Database	PostgreSQL + SQLAlchemy + Alembic
Font	Google Inter

Note on models: The assignment specified gemma2-9b-it and llama-3.3-70b-versatile. Both were decommissioned by Groq (October 2025 and August 2026 respectively). openai/gpt-oss-20b and openai/gpt-oss-120b are Groq's official recommended replacements, per their deprecation page (<https://console.groq.com/docs/deprecations>).

Features

Mandatory AI Tools

- 1. Log Complaint Tool** — Paste raw complaint text into the Copilot chat. The AI extracts all fields (product, batch, dates, customer, etc.), classifies the complaint, and generates a risk assessment.
- 2. Edit Complaint Tool** — Send a follow-up correction in natural language (e.g., "Sorry, the batch number is BMX240602 and affected quantity is 48 capsules"). The AI updates only the specified fields while preserving all other data, and re-runs the risk assessment.
- 3. Document Extraction Tool** — Upload a PDF complaint document. The AI extracts all fields from the PDF, populates the form, and generates a risk assessment. Corrections can still be made via natural language after extraction.

Bonus AI Features

- **AI Risk Classification** — Severity classification (Major, Critical) with suggested next action and initial risk assessment

- Duplicate Complaint Detection — Checks against existing complaints in the database and flags potential duplicates with similarity scores
- AI-Suggested Investigation Areas — Suggests potential root cause areas to investigate based on the complaint type
- CAPA Recommendation — Drafts Corrective and Preventive Actions automatically
- Complaint Completeness Checker — Identifies missing required fields before commit
- Complaint Summary — Generates a structured defect summary

LangGraph Pipeline

The AI workflow is a LangGraph StateGraph with distinct, independently testable nodes:

```

input_router
  |__(new)_____> extract_new _____|
  |__(correction)__> extract_correction __|
                                     v
                                merge_state
                                     v
                                classify_complaint
                                     v
                                assess_risk
                                     v
                                check_completeness
                                     v
                                detect_duplicates
                                     v
                                suggest_root_cause
                                     v
                                suggest_capa
                                     v
                                summarize
                                     v
                                finalize_response
                                     v
                                END

```

Each node reads from and writes to a shared ComplaintGraphState TypedDict. The graph is compiled once and reused across requests. The correction path re-runs the entire pipeline (classification, risk, CAPA) — not just field extraction — because a correction can change what the complaint fundamentally is.

Project Structure

```

AV0IA/
|-- frontend/                                # React + Redux frontend
|  |-- src/
|  |  |-- store/slices/
|  |  |  |-- complaintSlice.js             # Redux state + thunks
|  |  |-- services/
|  |  |  |-- api.js                        # API call functions
|  |  |-- components/                     # UI components
|  |  |-- types/
|  |  |  |-- complaint.js                 # Field definitions
|  |-- index.html
|  |-- vite.config.js                     # Vite + API proxy to :8000

```

```

|-- package.json
|-- backend/                                # FastAPI + LangGraph backend
|   |-- app/
|       |-- ai/
|           |-- graph.py                    # LangGraph wiring
|           |-- state.py                    # ComplaintGraphState
|           |-- groq_client.py              # Groq API wrapper
|           |-- nodes/
|               |-- input_router.py
|               |-- extraction.py
|               |-- merge_state.py
|               |-- classification.py
|               |-- risk_assessment.py
|               |-- completeness.py
|               |-- duplicate_detection.py
|               |-- bonus_nodes.py          # root cause, CAPA, summary
|               |-- finalize.py
|           |-- prompts/
|               |-- extraction.py
|       |-- api/
|           |-- complaints.py               # FastAPI routes
|       |-- services/
|           |-- complaint_service.py        # Business logic
|       |-- models/
|           |-- complaint.py                # SQLAlchemy models
|       |-- schemas/
|           |-- complaint.py                # Pydantic schemas
|       |-- database/
|           |-- session.py                  # DB session
|       |-- config.py                       # Settings (Groq, DB, CORS)
|       |-- main.py                         # FastAPI app
|-- requirements.txt
|-- .env.example
|-- demo_data/
|   |-- demo1_text_complaint.txt
|   |-- demo1_correction_message.txt
|   |-- demo2_correction_message.txt
|   |-- generate_demo_pdf.py
|   |-- zenith_life_sciences_CC-2026-00154.pdf
|-- .gitignore
|-- README.md

```

Setup & Installation

Prerequisites

- Node.js 18+ and npm
- Python 3.11+
- PostgreSQL (running locally)
- A Groq API key (create one at <https://console.groq.com/keys>)

Backend

```

cd backend

# Create and activate virtual environment

```

```
python -m venv venv
source venv/bin/activate      # Linux/Mac
# or: .\env\Scripts\activate  # Windows PowerShell

# Install dependencies
pip install -r requirements.txt

# Configure environment
cp .env.example .env
# Edit .env with your Groq API key and database URL

# Run database migrations
alembic upgrade head

# Start the backend
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend

# Install dependencies
npm install

# Start the dev server
npm run dev
```

The frontend runs on <http://localhost:5173> and proxies /api requests to the backend at <http://localhost:8000>.

Environment Variables

Backend (backend/.env)

```
GROQ_API_KEY=your_groq_api_key_here
GROQ_PRIMARY_MODEL=openai/gpt-oss-20b
GROQ_SECONDARY_MODEL=openai/gpt-oss-120b
DATABASE_URL=postgresql://user:password@localhost:5432/aivoa_complaints
APP_ENV=development
CORS_ORIGINS=http://localhost:5173,http://localhost:3000
MAX_UPLOAD_MB=10
```

Frontend (frontend/.env)

```
VITE_API_BASE_URL=/api
```

Demo Flow

Scenario 1: Text Complaint + Correction

- Click Reset Form
- Paste into the Copilot chat: "Apollo Pharmacy reported discolored capsules in Amoxicillin Capsules 500 mg. Batch number AMX240602. Manufacturing date March 2026. Expiry date February 2028. Please log this complaint."

- The form auto-populates with Amoxicillin data, severity classified as Major
- Send a correction: "Sorry, the batch number is BMX240602 and affected quantity is 48 capsules."
- The form updates batch number and quantity; risk assessment re-runs

Scenario 2: PDF Upload + Correction

- Click Reset Form
- Upload demo_data/zenith_life_sciences_CC-2026-00154.pdf
- The form populates with Metformin API data, severity classified as Critical
- Send a correction: "Correction: the batch number should read CHG260712A, and the affected quantity is actually 50 kg across 2 HDPE drums, not 1."
- The form updates; risk assessment and CAPA regenerate

API Endpoints

Method	Path	Description
POST	/api/complaints/process-text	Process pasted complaint text
POST	/api/complaints/process-pdf	Upload and process a PDF complaint
POST	/api/complaints/chat	Send a correction/update message
POST	/api/complaints/commit	Commit complaint to QMS ledger
GET	/api/complaints/{id}	Get full complaint state
GET	/api/complaints/{id}/audit-log	Get audit trail for a complaint
GET	/api/complaints	List all complaints

Key Design Decisions

- No manual form entry — All data is entered through the Copilot chat, as required by the assignment. The form is read-only, populated entirely by AI-structured responses.
- Full state response on corrections — The correction endpoint returns a complete ExtractionResponse (all fields, risk assessment, CAPA, duplicates), not just the changed fields. This ensures the frontend never shows stale AI-generated data after a correction.
- Human-in-the-loop merge — The merge node only overwrites fields the LLM explicitly returned. Unmentioned fields are preserved, so the user's previous data is never accidentally wiped.
- Audit trail — Every field change is logged with old and new values, providing full traceability for QMS compliance.
- Dual-model strategy — Extraction uses the faster gpt-oss-20b; risk assessment uses the stronger gpt-oss-120b with automatic fallback to the primary model.

Demo Data

All demo data is fictional, created for the AIVOA assignment:

- demo1_text_complaint.txt — Amoxicillin complaint text prompt
- demo1_correction_message.txt — Batch/quantity correction for Scenario 1
- demo2_correction_message.txt — Batch/quantity correction for Scenario 2
- generate_demo_pdf.py — Script to generate the Zenith Life Sciences PDF
- zenith_life_sciences_CC-2026-00154.pdf — Fictional pharma complaint PDF for document extraction demo

License

This project is created for the AIVOA AI Product Engineer internship application.